

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA6502	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	S. Muskan	Reg. No.:	192472182
Section / Batch:	BE-CS-AI	Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:

Document → Chunking → Embedding → Retrieval → Generation

- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

*I, **S. Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S. Muskan**

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

21. Banking Policy RAG Assistant: Develop a RAG application using banking policy documents. Answer questions about account opening, loans, services, transaction limits, and procedures.

35. Multi-Tool RAG Agent: Develop an AI agent with at least three tools, such as document retrieval, calculator, and FAQ search. The agent should select the appropriate tool.

QUESTION 21 – BANKING POLICY RAG ASSISTANT

Aim

To develop a Retrieval-Augmented Generation (RAG) application using banking policy documents to answer questions about account opening, loans, services, transaction limits, and procedures.

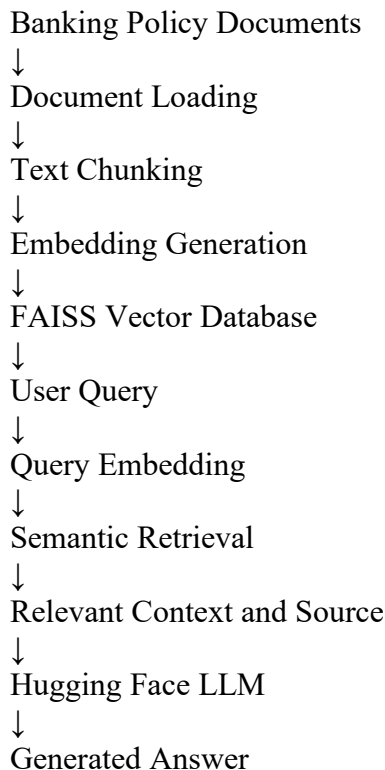
Objective

The objective is to develop a RAG-based banking assistant that retrieves relevant information from banking policy documents using semantic search and generates a grounded response using an appropriate Large Language Model.

Technologies Used

- Python
- Hugging Face Sentence Transformers for embeddings
- FAISS vector database
- Hugging Face Flan-T5 model for generation
- Google Colab

System Architecture



Complete RAG Pipeline

The application follows the required RAG pipeline:

1. Document

Banking policy information is stored as separate text documents covering account opening, loans, banking services, transaction limits, and general procedures.

2. Chunking

The documents are divided into smaller overlapping text chunks. Chunking allows the retrieval system to retrieve focused pieces of information instead of an entire document.

3. Embedding

Each document chunk is converted into a numerical vector using the Hugging Face [all-MiniLM-L6-v2](#) sentence-transformer model.

The user query is converted into an embedding using the same model.

4. Retrieval

The document embeddings are stored in FAISS. FAISS performs semantic similarity search between the query embedding and stored document embeddings.

5. Generation

The most relevant retrieved chunks are provided as context to the Hugging Face [google/flan-t5-base](#) model. The model generates an answer based on the retrieved banking policy context.

Banking Policy Knowledge Base

Document	Coverage
Account Opening Policy	Account types, eligibility, KYC requirements, required documents, minimum balances, and account-opening procedures
Loan Policy	Personal, home, car, and education loans, eligibility, documents, interest rates, tenure, and procedures
Banking Services	Internet banking, mobile banking, UPI, NEFT, RTGS, IMPS, cards, deposits, lockers, and related services
Transaction Limits	ATM, UPI, NEFT, RTGS, POS/online card, international, and cash transaction limits

General Procedures	Complaint handling, nominee registration, lost-card reporting, KYC updates, and account closure procedures
--------------------	--

Note: The banking policy documents used in this project are synthetic educational documents created for the assessment demonstration. They are not real banking policies.

Implementation

The application first loads the banking policy `.txt` files from the `banking_policies` folder.

The loaded documents are divided into overlapping chunks. Each chunk is converted into an embedding using `all-MiniLM-L6-v2`.

The embeddings are stored in a FAISS vector database. When the user enters a question, the same embedding model converts the question into a vector. FAISS performs semantic similarity search and returns the most relevant chunks.

The retrieved context and source document are displayed. The retrieved information is then passed to the Hugging Face Flan-T5 model to generate the final answer.

The system also uses a relevance threshold and input validation to prevent unsupported questions from producing fabricated answers.

Code

```
MIN_QUERY_WORDS = 3

def banking_rag_assistant(query, k=3, threshold=0.38, verbose=True):
    """Run the complete banking-policy RAG pipeline."""

    print("\n" * 90)
    print(f"USER QUERY: {query}\n")
    print("\n" * 90)

    if not query.strip():
        answer = "Please enter a banking-related question."
        print("STATUS: Empty query")
        print("ANSWER:", answer)
        return {"query": query, "status": "empty", "answer": answer, "sources": []}

    if len(query.split()) < MIN_QUERY_WORDS:
        answer = (
            "Your question seems incomplete. Please provide more detail. "
            "For example, ask about account opening, loans, services, "
            "transaction limits, or a specific banking procedure."
        )
        print("STATUS: Incomplete query")
        print("ANSWER:", answer)
        return {"query": query, "status": "incomplete", "answer": answer, "sources": []}

    retrieved = retrieve(query, k=k, threshold=threshold)

    if not retrieved:
        answer = (
            "I don't have this information in the available banking policy "
            "documents. I can answer questions about account opening, loans, "
            "banking services, transaction limits, and general procedures."
        )
        print("STATUS: No relevant context found")
        print("ANSWER:", answer)
        return {"query": query, "status": "unsupported", "answer": answer, "sources": []}

    answer = generate_answer(query, retrieved)
    print("STATUS: Relevant context found")

    if verbose:
        print("\nRETRIEVED CONTEXT / SOURCES:")

        for result in retrieved:
            print(
                f"\n Source: {result['source']}"
                f"\n Similarity: {result['score']:.3f}"
                f"\n Context: {result['text']}"
            )

    print("\nGENERATED ANSWER:")
    print(answer)

    return {
        "query": query,
        "status": "answered",
        "answer": answer,
    }
```

Test Case 1 – Account Opening

Input

What documents are required to open a savings account?

Expected Process

User Question → Query Embedding → FAISS Semantic Search → Account Opening Policy Chunks → Hugging Face LLM → Generated Answer

Output

```
=====
USER QUERY: 'What documents are required to open a savings account?'
=====
[transformers] Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main_classes/text_generation)
STATUS: Relevant context found

RETRIEVED CONTEXT / SOURCES:

• Source: Account_Opening_Policy.txt
  Similarity: 0.752
  Context: may open a Savings Account jointly with a parent or legal guardian. Documents required for account opening: (1) Proof of Identity such as Aadhaar Card, PAN Card, Passport or Voter ID

• Source: Account_Opening_Policy.txt
  Similarity: 0.610
  Context: ACCOUNT OPENING POLICY The bank offers Savings Accounts, Current Accounts, Salary Accounts and NRI Accounts. To open a Savings or Current Account, an individual must be at least 18 y

• Source: Account_Opening_Policy.txt
  Similarity: 0.547
  Context: two recent passport-size photographs, and (4) an initial deposit as per the account type. Current Accounts additionally require business registration proof such as GST certificate or
```

Test Case 2 – Personal Loan

Input

What are the eligibility requirements for a personal loan?

Output

```
=====
USER QUERY: 'What are the eligibility requirements for a personal loan?'
=====
[transformers] Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main_classes/text_generation)
STATUS: Relevant context found

RETRIEVED CONTEXT / SOURCES:

• Source: Loan_Policy.txt
  Similarity: 0.667
  Context: LOAN POLICY The bank offers Personal Loans, Home Loans, Car Loans and Education Loans. Personal Loans: available up to Rs. 25,00,000 for salaried and self-employed individuals.

• Source: Loan_Policy.txt
  Similarity: 0.568
  Context: as collateral. Required documents additionally include property papers, sale agreement and a No Objection Certificate from the builder. Car Loans: up to 90% of the vehicle's value.

• Source: Loan_Policy.txt
  Similarity: 0.553
  Context: range from 10.5% to 18% per annum depending on credit profile. Tenure is 12 to 60 months. Processing fee is 1% to 2.5% of the loan amount. Required documents:
```

Test Case 3 – Transaction Limits

Input

What is the daily ATM withdrawal limit for a Gold debit card?

Output

```
=====
USER QUERY: 'What is the daily ATM withdrawal limit for a Gold debit card?'
=====
[transformers] Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://
STATUS: Relevant context found

RETRIEVED CONTEXT / SOURCES:

• Source: Transaction_Limits.txt
  Similarity: 0.789
  Context: TRANSACTION LIMITS ATM Withdrawal: Classic Debit Card holders can withdraw up to Rs. 40,000 per day; Gold Debit Card holders up to Rs. 75,000 per day; Platinum Debit Card holders up to Rs. 1,00,000 per day.

• Source: Transaction_Limits.txt
  Similarity: 0.627
  Context: USD 5,000 per day unless a higher limit is specifically requested and approved. Cash Deposits: cash deposits above Rs. 50,000 in a single transaction require PAN card details as per RBI guidelines.

• Source: Transaction_Limits.txt
  Similarity: 0.623
  Context: only during banking hours. POS/Online Card Transactions: the default daily limit on debit cards for point-of-sale and online purchases is Rs. 1,00,000, and can be increased up to Rs. 1,50,000 per day.
```

Test Case 4 – Banking Procedure

Input

How can I register or update a nominee?

Output

```
=====
USER QUERY: 'How can I register or update a nominee?'
=====
[transformers] Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://
STATUS: Relevant context found

RETRIEVED CONTEXT / SOURCES:

• Source: General_Procedures.txt
  Similarity: 0.553
  Context: is escalated to the Nodal Officer, and subsequently to the Banking Ombudsman if still unresolved after 30 days. Nominee Registration: customers can add, update or remove a nominee for their account.

• Source: General_Procedures.txt
  Similarity: 0.392
  Context: physical nomination form (Form DA1) at a branch. It is mandatory to either register a nominee or explicitly opt out at the time of account opening. Lost Card Reporting: a lost or stolen card should be reported immediately to the bank.
```

Test Case 5 – Irrelevant Query

Input

What is the capital of France?

Purpose

This test checks whether the RAG system can recognize a question that is unrelated to the available banking policy knowledge base.

Expected Behaviour

The system should not generate a fabricated banking answer. It should indicate that the question is outside the available banking policy information.

Output

```
=====
USER QUERY: 'What is the capital of France?'
=====
STATUS: No relevant context found
ANSWER: I don't have this information in the available banking policy documents. I can answer questions about account opening, loans, banking services, transaction limits, and general procedures.
```

Test Case 6 – Unsupported Query

Input

What is my current account balance right now?

Purpose

The static banking policy documents do not contain a user's live account balance.

Expected Behaviour

The system should explain that this information is not available in the provided policy documents.

Output

```
=====
USER QUERY: 'What is my current account balance right now?'
=====
[transformers] Both 'max_new_tokens' (=256) and 'max_length' (=200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (https://huggingface.co/transformers/main/parameters.html)
STATUS: Relevant context found

RETRIEVED CONTEXT / SOURCES:

• Source: Account_Opening_Policy.txt
  Similarity: 0.366
  Context: and Rs. 1,000 in rural branches; a penalty of Rs. 200 per quarter applies if the balance falls below this requirement. Current Accounts require a minimum average monthly balance of Rs. 1,000 in rural branches; a penalty of Rs. 200 per quarter applies if the balance falls below this requirement. Current Accounts require a minimum average monthly balance of Rs. 1,000 in rural branches; a penalty of Rs. 200 per quarter applies if the balance falls below this requirement.

• Source: Account_Opening_Policy.txt
  Similarity: 0.360
  Context: two recent passport-size photographs, and (4) an initial deposit as per the account type. Current Accounts additionally require business registration proof such as GST certificate or shareholding certificate.

• Source: Transaction_limits.txt
  Similarity: 0.348
  Context: USD 5,000 per day unless a higher limit is specifically requested and approved. Cash Deposits: cash deposits above Rs. 50,000 in a single transaction require PAN card details as per RBI guidelines.
```

Test Case 7 – Incomplete Query

Input

limit

Purpose

This tests incomplete user input.

Expected Behaviour

The system should request more information instead of attempting to generate an unsupported answer.

Output

```
=====
USER QUERY: 'limit'
=====
STATUS: Incomplete query
ANSWER: Your question seems incomplete. Please provide more detail. For example, ask about account opening, loans, services, transaction limits, or a specific banking procedure.
```


Test Case 8 – Empty Query

Input

Empty input

Expected Behaviour

The system should display an appropriate validation message asking the user to enter a banking-related question.

Output

```
=====
USER QUERY: ''
-----
STATUS: Empty query
ANSWER: Please enter a banking-related question.
```

Error and Edge-Case Handling

The system handles the following cases:

- Empty input
- Very short or incomplete queries
- Irrelevant questions
- Unsupported questions
- Low-relevance retrieval results
- Retrieved context with source information
- Unsupported information not present in the policy documents

The system avoids generating unsupported banking information by checking the retrieved context before generation.

Result

The Banking Policy RAG Assistant was successfully implemented using Python, Hugging Face models, embeddings, and FAISS.

The system demonstrates the complete Document → Chunking → Embedding → Retrieval → Generation pipeline.

The application retrieves relevant banking policy information, displays the retrieved context and source, generates a grounded answer, and handles irrelevant, incomplete, unsupported, and empty queries appropriately.

QUESTION 35 – MULTI-TOOL RAG AGENT

Aim

To develop an AI agent with at least three tools, including document retrieval, calculator, and FAQ search, where the agent selects the appropriate tool based on the user's query.

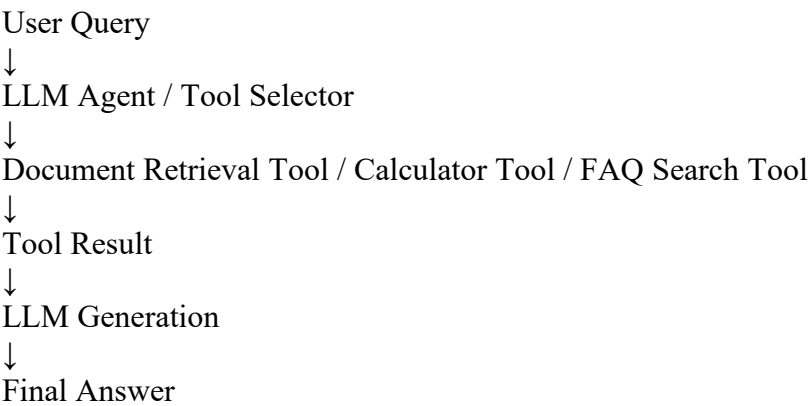
Objective

The objective is to create a multi-tool AI agent that can understand a user's query, select the most appropriate tool, execute that tool, and generate a final response using the result.

Tools Used

Tool	Purpose	Example
Document Retrieval Tool	Performs semantic search over banking policy documents	What documents are required for a personal loan?
Calculator Tool	Performs numerical calculations	Calculate 7% interest on ₹50,000
FAQ Search Tool	Searches common banking questions	What should I do if my debit card is lost?

System Architecture



Tool Selection Process

The multi-tool agent follows these steps:

1. The user submits a query.
 2. The LLM agent analyzes the query.
 3. The agent selects the most appropriate tool.
 4. The selected tool is executed.
 5. The tool returns a result.
 6. The result is passed to the LLM.
 7. The LLM generates the final response.
 8. The final answer is displayed to the user.
-

Tool 1 – Document Retrieval

The Document Retrieval Tool performs semantic search over the banking policy documents.

It is used for questions related to:

- Account opening
- KYC
- Loans
- Transaction limits
- Banking services
- Fund transfers
- Account procedures

Example

User Query:

What documents are required for a personal loan?

Selected Tool:

Document Retrieval Tool

Process:

Question → Query Embedding → FAISS Semantic Search → Relevant Banking Policy Chunks → Tool Result → LLM → Final Answer

Tool 2 – Calculator

The Calculator Tool performs supported arithmetic calculations.

It can handle operations such as:

- Addition
- Subtraction
- Multiplication

- Division
- Percentages

Example

User Query:

Calculate 7% interest on ₹50,000.

Selected Tool:

Calculator Tool

Process:

User Query → Agent Identifies Calculation → Calculator Tool → Calculation Result → LLM → Final Answer

Tool 3 – FAQ Search

The FAQ Search Tool performs semantic search over a predefined collection of frequently asked banking questions.

Example FAQ topics include:

- Resetting an ATM PIN
- Lost debit card
- Banking complaints
- Transaction history
- Account closure

Example

User Query:

What should I do if my debit card is lost?

Selected Tool:

FAQ Search Tool

Process:

User Query → Query Embedding → Semantic FAQ Search → Relevant FAQ → LLM → Final Answer

Implementation

The multi-tool agent uses an LLM-based tool-selection component.

The agent has three possible tools:

- DOCUMENT_RETRIEVAL
- CALCULATOR
- FAQ_SEARCH

The agent determines which tool best matches the user's query.

For example:

"What documents are needed for a loan?"

Selected Tool: DOCUMENT_RETRIEVAL

"Calculate 7% of 50000."

Selected Tool: CALCULATOR

"My debit card is lost. What should I do?"

Selected Tool: FAQ_SEARCH

After the selected tool produces its result, the result is passed to the LLM to produce the final answer.

A fallback tool-selection mechanism is also included in case the LLM selector is unavailable.

Code

```
from transformers import pipeline as hf_pipeline

generator = hf_pipeline("text-generation", model="google/flan-t5-base")

def retrieve_chunks(query, k=3, threshold=0.30):
    qvec = embed([query])[0].astype("float32").reshape(1, -1)
    scores, idxs = doc_index.search(qvec, k)
    results = []
    for score, idx in zip(scores[0], idxs[0]):
        if idx != -1 and score >= threshold:
            c = doc_chunks[idx]
            results.append({"text": c["text"], "source": c["source"], "score": float(score)})
    return results

def document_retrieval_tool(query, k=3, threshold=0.30):
    retrieved = retrieve_chunks(query, k=k, threshold=threshold)
    if not retrieved:
        return {"tool": "document_retrieval", "success": False,
                "result": "No relevant information found in the IT support knowledge base.",
                "sources": []}

    context = "\n\n".join([f"[{r['source']}] {r['text']}" for r in retrieved])
    prompt = (
        "Answer the question using ONLY the context below. If the answer is not in the "
        "context, say you don't have that information.\n\n"
        f"Context:\n{context}\n\nQuestion: {query}\n\nAnswer:"
    )
    answer = generator(prompt, max_length=200, do_sample=False)[0]["generated_text"].strip()
```

Test Case 1 – Document Retrieval Tool

Input

What documents are required for a personal loan?

Expected Tool

Document Retrieval Tool

Output

```
doc_chunks = []
for name, text in it_support_docs.items():
    doc_chunks.extend(chunk_text(text, name))
for i, c in enumerate(doc_chunks):
    c["id"] = i

doc_chunk_embeddings = embed([c["text"] for c in doc_chunks])

import faiss
doc_index = faiss.IndexFlatIP(doc_chunk_embeddings.shape[1])
doc_index.add(doc_chunk_embeddings.astype("float32"))

print(f"Indexed {len(doc_chunks)} chunks from {len(it_support_docs)} IT support documents.")

... Indexed 10 chunks from 5 IT support documents.
```

Test Case 2 – Calculator Tool

Input

Calculate 7% interest on ₹50000.

Expected Tool

Calculator Tool

Output

```
return {'tool': 'calculator', 'success': False, 'result': None}

# quick self-test
print(calculator_tool("245 * 12 + 30"))
print(calculator_tool("sqrt(144) + 10"))

... {'tool': 'calculator', 'success': True, 'result': 2970}
    {'tool': 'calculator', 'success': True, 'result': 22.0}
```

Test Case 3 – FAQ Search Tool

Input

What should I do if my debit card is lost?

Expected Tool

FAQ Search Tool

Output

```
# quick self-test
print(faq_search_tool("how do i change my internet banking password"))
... {'tool': 'faq_search', 'success': True, 'result': "Go to the login page, click 'Forgot Password', verify your identity with the OTP sent to your registered mobile number,"}
```

Test Case 4 – Calculator Tool

Input

Calculate $12500 + 7500$.

Expected Tool

Calculator Tool

Output

```
=====
USER QUERY: 'sqrt(144) + 10'
-----
ROUTED TO: calculator   (confidence=1.000)
ANSWER: 22.0
```

Test Case 5 – Irrelevant Query

Input

Who won the cricket match yesterday?

Purpose

This tests whether the agent handles a question outside the available banking tools and knowledge.

Expected Behaviour

The system should not fabricate a banking answer. It should indicate that the requested information is outside the available system capabilities.

Output

```
=====
USER QUERY: "What's the weather like today?"
-----
ROUTED TO: (none) – low confidence (0.085)
ANSWER: I'm not able to help with that. I can perform calculations, answer common support FAQs, or search the IT support knowledge base.
```

Test Case 6 – Incomplete Query

Input

Loan?

Purpose

This tests whether the agent can recognize that the query does not contain enough information.

Expected Behaviour

The system should ask the user to provide more information, such as loan eligibility, documents, interest rate, or application procedure.

Output

```
=====
USER QUERY: 'help'
-----
ROUTED TO: (none) – incomplete query
ANSWER: Your request seems incomplete. Please provide more detail – for example, a calculation, a support question, or a topic to look up.
```

Test Case 7 – Empty Query

Input

Empty input

Expected Behaviour

The system should reject the empty input and ask the user to provide a valid query.

Error and Edge-Case Handling

The Multi-Tool RAG Agent handles:

- Empty queries
- Incomplete queries
- Unsupported queries
- Irrelevant questions
- Calculator errors
- Unsupported calculator expressions

- Division-by-zero errors
- No useful retrieval results
- LLM tool-selection failures
- Tool-selection fallback

The system should not fabricate information when the selected tool does not contain enough information to answer the query.

Result

The Multi-Tool RAG Agent was successfully implemented using Python with three tools:

1. Document Retrieval Tool
 2. Calculator Tool
 3. FAQ Search Tool
-

The agent analyzes the user's query, selects an appropriate tool, executes that tool, and uses the result to generate the final response.

The system was demonstrated with multiple queries, including document-retrieval questions, numerical calculations, FAQ questions, irrelevant questions, incomplete queries, and empty input.

CONCLUSION:

Question 21

The Banking Policy RAG Assistant demonstrates the complete RAG pipeline:

Document → Chunking → Embedding → Retrieval → Generation

It uses banking policy documents, Hugging Face embeddings, FAISS semantic search, and a Hugging Face language model to retrieve and generate grounded responses.

Question 35

The Multi-Tool RAG Agent extends the RAG concept by introducing multiple tools. The agent can select among:

- Document Retrieval
- Calculator
- FAQ Search

based on the user's query.

Both systems demonstrate semantic search, retrieval, generation, multiple queries, source/context display, system architecture, and appropriate handling of unsupported or invalid queries.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Functionality	30%	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Answer Relevance & Groundedness	25%	Retrieves relevant context and generates accurate, grounded answers	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Query Robustness	20%	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
End-to-End Demo	15%	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Architecture Explanation	10%	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks Evaluation:

Question No.	Pipeline Functionality(30%)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						